SQL Normalization and Denormalization

SQL Normalization

Normalization in SQL is a database design technique that organizes tables in a manner that reduces redundancy and dependency. It involves dividing a database into two or more tables and defining relationships between them to achieve a more efficient database structure.

Characteristics

- **Reduces Redundancy:** Avoids duplication of data.
- **Improves Data Integrity:** Ensures data accuracy and consistency.
- **Database Design:** Involves creating tables and establishing relationships through primary and foreign keys.

Example: Customer Orders Database

Original Table (Before Normalization)

Imagine a single table that stores all customer orders:

Customer ID	Customer Name	Customer Address	Order ID	Order Date	Product
001	John Doe	123 Apple St.	1001	2021-08-01	Laptop
001	John Doe	123 Apple St.	1002	2021-08-05	Phone
002	Jane Smith	456 Orange Ave.	1003	2021-08-03	Tablet

This table has redundancy (notice how customer details are repeated) and is not normalized.

After Normalization

To normalize this, we would split it into two or more tables to reduce redundancy.

Customers Table (1NF, 2NF, 3NF)

Customer ID	Customer Name	Customer Address
001	John Doe	123 Apple St.
002	Jane Smith	456 Orange Ave.

Orders Table (1NF, 2NF, 3NF)

Order ID	Order Date	Product	Customer ID
1001	2021-08-01	Laptop	001
1002	2021-08-05	Phone	001
1003	2021-08-03	Tablet	002

In the normalized structure, we've eliminated redundancy (each customer's details are listed only once) and established a relationship between the two tables via `CustomerID`.

Levels (Normal Forms)

- **1NF (First Normal Form):** Data is stored in atomic form with no repeating groups.
- **2NF (Second Normal Form):** Meets 1NF and has no partial dependency on any candidate key.
- **3NF (Third Normal Form):** Meets 2NF and has no transitive dependency.

Use Cases

- Ideal for complex systems where data integrity is critical, like financial or enterprise applications.

SQL Denormalization

Denormalization, on the other hand, is the process of combining tables to reduce the complexity of database queries. This can introduce redundancy but may lead to improved performance by reducing the number of joins required.

Characteristics

- **Increases Redundancy:** May involve some data duplication.
- **Improves Query Performance:** Reduces the complexity of queries by reducing the number of joins.
- **Data Retrieval:** Optimized for read-heavy operations.

Denormalization Example

Denormalization would involve combining these tables back into a single table to optimize read performance. Taking the above table:

Denormalized Orders Table

Customer ID	Customer Name	Customer Address	Order ID	Order Date	Product
001	John Doe	123 Apple St.	1001	2021-08-01	Laptop
001	John Doe	123 Apple St.	1002	2021-08-05	Phone
002	Jane Smith	456 Orange Ave.	1003	2021-08-03	Tablet

Here, we're back to the original structure. The benefit of this denormalized table is that it can make queries faster since all the information is in one place, reducing the need for JOIN operations. However, the downside is the redundancy of customer information, which can take up more space and potentially lead to inconsistencies if not managed properly.

When to Use

- In read-heavy database systems where query performance is a priority.
- In systems where data changes are infrequent and a slightly less normalized structure doesn't compromise data integrity.

Key Differences

1. **Purpose:**
   - Normalization aims to minimize data redundancy and improve data integrity.
   - Denormalization aims to improve query performance.
2. **Data Redundancy:**
   - Normalization reduces redundancy.
   - Denormalization may introduce redundancy.
3. **Performance:**
   - Normalization can lead to a larger number of tables and more complex queries, potentially affecting read performance.
   - Denormalization can improve read performance but may affect write performance due to data redundancy.
4. **Complexity:**
   - Normalization increases the complexity of the write operations.
   - Denormalization simplifies read operations but can make write operations more complex.

Conclusion

- **Normalization** is about reducing redundancy and improving data integrity but can lead to more complex queries.
- **Denormalization** simplifies queries but at the cost of increased data redundancy and potential maintenance challenges.

The choice between the two depends on the specific requirements of your database system, considering factors like the frequency of read vs. write operations, and the importance of query performance vs. data integrity.

← Previous
Real-World Examples And Case Studies

Next →
In-Memory Database Vs. On-Disk Database

☐ Mark as Completed